

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ ГОСУДАРСТВЕННОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «САМАРСКИЙ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ имени
академика С.П. КОРОЛЁВА»

КАФЕДРА «ТЕХНИЧЕСКАЯ КИБЕРНЕТИКА»

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Объектно-ориентированное программирование»

ЛАБОРАТОРНАЯ РАБОТА №7

Студент Семенова В. Г.

Группа 6301-030301D

Руководитель Борисов Д. С.

Оценка _____

Оглавление	
Задание 1	3
Задание 2	4
Задание 3	6

Задание 1

Обновили интерфейс «TabulatedFunctions», добавив родительский тип `Iterable<FunctionPoint>`. Теперь все реализации `TabulatedFunction` можно использовать в `for-each`. В «`ExternalizableArrayTabulatedFunction`», «`ExternalizableLinkedListTabulatedFunction`», «`ArrayTabulatedFunction`», «`LinkedListTabulatedFunction`» добавили метод `iterator()`, возвращающий анонимный класс итератора. Класс `iterator()` содержит метод проверки наличия следующего элемента, метод получения следующего элемента, метод удаления элементов, который выбрасывает исключение, так как удаление не поддерживается

```
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode currentNode = head.nextEl;
        private boolean started = false;

        public boolean hasNext() {
            return currentNode != head && (!started || currentNode != head.nextEl);
        }

        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("No more elements in the
tabulated function");
            }

            FunctionPoint point = new FunctionPoint(currentNode.point);
            currentNode = currentNode.nextEl;
            started = true;
            return point;
        }

        public void remove() {
            throw new UnsupportedOperationException("Remove operation is not
supported");
        }
    };
}
```

Проверка корректной работы добавлена в класс «Main»:

```
// 1. Проверка ExternalizableArrayTabulatedFunction
System.out.println("1. ExternalizableArrayTabulatedFunction:");
TabulatedFunction f1 = new
ExternalizableArrayTabulatedFunction(testPoints);
for (FunctionPoint p : f1) {
```

```
System.out.println(" " + p);  
}
```

```
=== Задание 1. ПРОСТАЯ ПРОВЕРКА ИТЕРАТОРОВ ===
```

```
1. ExternalizableArrayTabulatedFunction:  
  (1,000; 2,000)  
  (2,000; 4,000)  
  (3,000; 6,000)  
  (4,000; 8,000)  
  (5,000; 10,000)  
  
2. ExternalizableLinkedListTabulatedFunction:  
  (1,000; 2,000)  
  (2,000; 4,000)  
  (3,000; 6,000)  
  (4,000; 8,000)  
  (5,000; 10,000)  
  
3. ArrayTabulatedFunction (обычная):  
  (1,000; 2,000)  
  (2,000; 4,000)  
  (3,000; 6,000)  
  (4,000; 8,000)  
  (5,000; 10,000)  
  
4. LinkedListTabulatedFunction (обычная):  
  (1,000; 2,000)  
  (2,000; 4,000)  
  (3,000; 6,000)  
  (4,000; 8,000)  
  (5,000; 10,000)  
  
=== ПРОВЕРКА ЗАВЕРШЕНА ===
```

Задание 2

Создали систему фабрик, которая позволяет динамически изменять тип создаваемых функций. Создали интерфейс фабрики, который содержит три перегруженных (с одинаковым именем, но разными параметрами) метода.

```
package functions;  
  
public interface TabulatedFunctionFactory {  
  
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int  
pointsCount);  
  
    TabulatedFunction createTabulatedFunction(double leftX, double rightX,  
double[] values);  
  
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points);  
}
```

В классы «ExternalizableArrayTabulatedFunction:», «ExternalizableLinkedListTabulatedFunction», «ArrayTabulatedFunction», «LinkedListTabulatedFunction» добавили вложенные статические класс-фабрики, которые создают объекты данных типов разными способами.

```
public static class ExternalizableArrayTabulatedFunctionFactory implements
TabulatedFunctionFactory {

    public TabulatedFunction createTabulatedFunction(double leftX, double
rightX, int pointsCount) {
        return new ExternalizableArrayTabulatedFunction(leftX, rightX,
pointsCount);
    }

    public TabulatedFunction createTabulatedFunction(double leftX, double
rightX, double[] values) {
        return new ExternalizableArrayTabulatedFunction(leftX, rightX, values);
    }

    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ExternalizableArrayTabulatedFunction(points);
    }
}
```

Методы создают объекты разными способами, вызывая разные конструкторы класса. Также обновили метод tabulate() в классе «TabulatedFunction», теперь он использует фабрику вместо создания «ArrayTabulatedFunction»

```
public static TabulatedFunction tabulate(Function function, double leftX, double
rightX, int pointsCount) {
    if (leftX >= rightX || pointsCount < 2) {
        throw new IllegalArgumentException("Некорректные параметры");
    }

    //Используем фабрику вместо new ArrayTabulatedFunction
    TabulatedFunction tabulatedFunction =
        createTabulatedFunction(leftX, rightX, pointsCount);

    double step = (rightX - leftX) / (pointsCount - 1);
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        double y = function.getFunctionValue(x);
        tabulatedFunction.setPointY(i, y);
    }

    return tabulatedFunction;
}
```

Корректная работа фабрик добавлена в «Main»:

```
Задание 2. Проверка фабрик:  
1. ArrayTabulatedFunction  
2. LinkedListTabulatedFunction  
3. ArrayTabulatedFunction  
4. ExternalizableArrayTabulatedFunction  
5. ExternalizableLinkedListTabulatedFunction
```

Задание 3

Добавили рефлексивные методы в класс «TabulatedFunctions», которые принимают класс как параметр и параметры конструктора. В методах находим конструктор переданного класса и создаем объект через этот конструктор. Теперь мы можем создавать объект, не зная заранее класс, а передавая его как параметр.

```
// Метод 1: Создать функцию из интервала и количества точек  
public static TabulatedFunction createTabulatedFunction(  
    Class<? extends TabulatedFunction> clazz,  
    double leftX, double rightX, int pointsCount) {  
  
    try {  
        // Найти конструктор (double, double, int)  
        Constructor<? extends TabulatedFunction> constructor =  
            clazz.getConstructor(double.class, double.class, int.class);  
  
        // Создать объект  
        return constructor.newInstance(leftX, rightX, pointsCount);  
  
    } catch (Exception e) {  
        throw new IllegalArgumentException("Ошибка создания функции", e);  
    }  
}
```

Корректная работа реализации рефлексии добавлена в «Main»:

Задание 3 Тестирование рефлексии:

1. ArrayTabulatedFunction
{(0,000; 0,000), (5,000; 0,000), (10,000; 0,000)}
2. ArrayTabulatedFunction
{(0,000; 0,000), (10,000; 10,000)}
3. LinkedListTabulatedFunction
{(0,000; 0,000), (10,000; 10,000)}
4. LinkedListTabulatedFunction
Первые 3 точки:
(0,000, 0,000)
(0,314, 0,309)
(0,628, 0,588)

Изменения: Добавили extends Cloneable к интерфейсу TabulatedFunction.
Добавили реализацию через фабрику и перегруженные методы чтения через рефлексияю.